SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5116

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage**:** 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Debugging & Optimization Task

Code of Conduct:

I V .Sandeep Reddy(192325096)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
V .Sandeep Reddy

Annexure A
Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures.
B. Refactor the code to minimize redundant computations and memory usage.
C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation
A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange
A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation
A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.
B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Performance Analysis of Block Cipher Modes of Operation

1. Debugging Incorrect Padding and IV Handling (Root Causes & Solutions)

- **Analysis of Padding Failures (PKCS#7):**
  - *Root Cause:* Block ciphers like AES operate on fixed-size blocks (typically 128 bits / 16 bytes). When the plaintext length is not an exact multiple of the block size, padding must be added. In many scripts, incorrect padding implementation arises from manual byte calculations where the trailing bytes do not adhere to standard PKCS#7 padding rules (e.g., failing to pad with bytes representing the total number of padding bytes added). During decryption, stripping arbitrary bytes or failing to check the padding value leads to corrupted data or immediate decryption crashes.
  - *Systematic Fix:* Implement strict PKCS#7 validation checks. The padding value must equal the number of padding bytes appended. During decryption, the last byte determines how many bytes to strip, and a validation pass must verify that all stripped padding bytes match this value to prevent padding oracle vulnerabilities.
- **Analysis of Initialization Vector (IV) Handling:**
  - *Root Cause:* Modes such as CBC, CFB, and OFB require an Initialisation Vector (IV) to ensure semantic security (i.e., identical plaintexts yield different ciphertexts). Decryption failures occur when:
    1. A fresh, random IV is not generated for each encryption session.
    2. The IV is omitted from the ciphertext package, making it impossible for the decryption routine to reconstruct the initial state.
    3. Static or predictable IVs are used, which exposes the system to pattern analysis.
  - *Systematic Fix:* Ensure cryptographically secure pseudo-random generation of IVs (using libraries like os.urandom or secrets) equal in size to the cipher's block length (16 bytes for AES). The IV must be securely prepended or appended to the raw ciphertext byte string during transmission so that the decryption module can extract it cleanly before processing blocks.

2. Code Refactoring to Minimize Redundant Computations and Memory Usage

- **Chunk-Based Streaming Architecture:**
  - *Inefficiency:* Naive Python scripts often load entire multi-megabyte or gigabyte files directly into RAM (file.read()), causing memory exhaustion or severe latency spikes.
  - *Optimization:* Refactor the code to process data files using fixed-size buffers or generators (e.g., reading files in chunks of 4KB or 8KB blocks). This bounds memory consumption to $O(1)$ space complexity regardless of total input size.

- **Eliminating Loop Redundancies & Pre-computing Keys:**
    - *Inefficiency:* Re-instantiating the cipher object or re-running key schedule expansion inside block iteration loops creates massive processing overhead.
    - *Optimization:* Move key expansion and cipher context initialization entirely *outside* of the file/data processing loop. Instantiate the cipher object once per session, allowing the underlying C/C++ backend to reuse the expanded key schedule across all data blocks.

3. In-Depth Performance Comparison and Optimization for Real-Time Secure Communication

To determine the most efficient configuration for real-time secure communication, the script analyzes the following operational modes:

- **Electronic Codebook (ECB) Mode:**
    - *Performance:* High speed and fully parallelizable.
    - *Security Flaw:* Highly insecure; identical plaintext blocks result in identical ciphertext blocks, leaking structural patterns of the underlying data. Completely unsuitable for secure communication.
- **Cipher Block Chaining (CBC) Mode:**
    - *Performance:* Moderate latency. Encryption is inherently sequential because each ciphertext block depends on the previous block ($C_i = E_k(P_i \oplus C_{i-1})$), meaning it cannot be parallelized on multi-core processors. However, decryption can be parallelized.
    - *Security:* Provides good security when paired with proper padding and unique IVs.
- **Cipher Feedback (CFB) & Output Feedback (OFB) Modes:**
    - *Performance:* Transform block ciphers into stream ciphers, allowing data to be processed in smaller units, which reduces buffering latency.
    - *Security:* Effective for streaming data, though error propagation in CFB can corrupt subsequent blocks during transmission errors.
- **Optimal Recommendation for Real-Time Systems:**
    - While standard script requirements evaluate basic modes, migrating or optimizing toward **Counter (CTR) Mode** or **Galois/Counter Mode (GCM)** yields maximum efficiency. CTR mode turns the block cipher into a parallel stream cipher where block encryption operations are completely independent of one another. This enables full multi-core CPU parallelization, minimizes latency, eliminates padding overhead entirely, and provides optimal throughput for high-speed, real-time secure data transmission